



Seminar paper in the field of Mathematics and Natural Sciences
Bachelor's program in Applied Mathematics

From Ideal Points to Robot Joints

Computational geometry and conformal geometric algebra

Yana Osinchuk

September 23, 2026

Declaration

I affirm that I have independently authored this seminar paper and have not used sources or aids other than those explicitly mentioned. Any directly quoted or paraphrased passages from other works are appropriately acknowledged.

The Python programs and written implementation that serve as part of the source material (Section 3) were produced by me. Generative AI tools were used as an aid for language editing and for reviewing the program code; all derivations, programs, and numerical results were checked by me.

Darmstadt, September 23, 2026

Place, date

Osinchuk Yana

Summary

Many elementary geometry programs are written as sequences of coordinate formulas. Such programs may produce a correct drawing, yet the reason why a construction works — and where it can fail — remains hidden. This paper studies a different design principle: Euclidean objects are lifted into algebraic models in which a construction is expressed by joins, meets, and duality. Homogeneous coordinates in $\mathbb{P}^2$ provide a closed incidence calculus for planar points and lines, including points at infinity. Conformal geometric algebra (CGA) $\mathcal{G}_{4,1}$ extends the same idea to spheres, planes, circles, and point pairs in $\mathbb{R}^3$.

Six course exercises are reorganised into one computational study: a circumcircle, a perpendicular constructed through an ideal point, the isosceles apices on a parallel line, a tripod obtained from three spheres, and a two-link and a three-link arm. The paper contributes (i) a six-stage pipeline — encode, construct, intersect, factor, select, validate — that separates algebraic intersection from application policy; (ii) exact statements with proofs for the number of isosceles apices, for the point-pair factorisation used in the CLUCalc programs, and for the three-link construction, which turns out to be an isosceles trapezoid; and (iii) an audit of the original programs, in which every CLUCalc construction is re-executed in an independent NumPy implementation of $\mathcal{G}_{4,1}$.

The experiments confirm the geometric constraints to machine precision: the maximum residuals are 3.42×10^{-14} for 5,000 random projective configurations and 4.44×10^{-16} and 4.44×10^{-16} for 10,000 two-link and 10,000 three-link targets. The experiments also separate two kinds of singularity. *Geometric* singularities, such as collinear points or a fully stretched arm, are intrinsic: circumcentres and elbows become ill-conditioned in every formulation. *Representation* singularities are artefacts of a particular construction: the original three-link program breaks down at target distance $d = 1$, although the arm is perfectly regular there, and its error grows like $\varepsilon_{\text{mach}}/|d - 1|$. A reflection-based reformulation removes this defect completely. The audit further shows why the submitted tripod needs its fourth leg: the apex projects outside the support triangle (stability margin -0.669), and only the outer foot restores static stability (margin $+0.304$).

The underlying projective and conformal models are established theory; no claim is made that the algebra itself is new. The contribution is the synthesis of the source constructions into one layered method, the precise statements about their solution sets, and a reproducible, residual-based evaluation.

Keywords: computational geometry; homogeneous coordinates; projective geometry; conformal geometric algebra; inverse kinematics; geometric constraints; numerical conditioning; static stability.

Contents

Declaration	i
Summary	ii
Notation	v
1 Introduction	1
2 Mathematical Background and Related Work	2
2.1 Why lift a geometric problem?	2
2.2 Geometric representations in robot kinematics	2
2.3 Algebraic correctness and numerical robustness	3
3 Source Constructions and Study Design	3
4 Projective Construction Layer	4
4.1 Points, lines, joins, and meets	4
4.2 Circumcentre from two perpendicular bisectors	5
4.3 Perpendicular transport through an ideal point	5
4.4 Isosceles apices as intersections of loci	6
5 Conformal Geometric Algebra Layer	7
5.1 Embedding Euclidean points	7
5.2 Tripod from a three-sphere meet	9
5.3 Two-link inverse kinematics	10
5.4 Three-link arm: an isosceles trapezoid	11
6 A Unified Construction Pipeline	12
6.1 Type before coordinates	13
6.2 Branch policy as an explicit function	13
6.3 Residuals as contracts	14
6.4 Geometric versus representation singularities	14
7 Experimental Methodology	15
7.1 Implementation and reproducibility	15
7.2 Experiment 1: projective covariance	15
7.3 Experiment 2: constraints of the closed forms	15
7.4 Experiment 3: tripod and support polygon	16

7.5	Experiment 4: conditioning near singularities	16
7.6	Experiment 5: executing the CLUCalc programs	16
8	Results	17
8.1	Constraint validation	17
8.2	Numerical tripod and static stability	17
8.3	Behaviour near degeneracy	18
8.4	Audit of the source programs	19
9	Discussion	20
9.1	What is genuinely unified?	20
9.2	The value of visible multiplicity	20
9.3	Where the algebra does not help automatically	20
9.4	Improvements for production implementations	21
9.5	Threats to validity	22
10	Conclusion	22
A	Selected Derivations	23
A.1	Why the projective perpendicular construction works	23
A.2	Linear reduction of the three-sphere intersection	23
A.3	Two-circle intersection formula	23
A.4	Proof of Lemma 1	23
A.5	Proof of Proposition 2	24
A.6	Error growth near the representation singularity	24
B	Revised Source Programs	24
B.1	Hausaufgabe 1: tripod with fourth support	25
B.2	Hausaufgabe 2: two-link arm	26
B.3	Hausaufgabe 3: three-link arm	27
B.4	Python programs	28
C	Reproducibility	29
	References	30
	List of Figures	32
	List of Tables	33

Notation

Symbol	Meaning
$\mathbb{P}^2$	Real projective plane; points and lines are non-zero triples up to scale.
$p \times q$	Cross product: join of two points or meet of two lines in $\mathbb{P}^2$.
$\mathcal{G}_{4,1}$	Conformal geometric algebra of $\mathbb{R}^3$ (Clifford algebra of $\mathbb{R}^{4,1}$, $2^5 = 32$ basis blades).
e_0, e_∞	Null vectors for the origin and for infinity, $e_0 \cdot e_\infty = -1$ (CLUCalc: e0 , einf).
$X = \mathcal{C}(x)$	Normalised conformal point of $x \in \mathbb{R}^3$, i.e. $X^2 = 0$ and $X \cdot e_\infty = -1$.
$\wedge, \cdot, A^*$	Outer product, inner product, and dual $A^* = AI^{-1}$ with $I = e_1 e_2 e_3 e_+ e_-$.
OPNS / IPNS	Outer-product and inner-product null-space representation of an object.
$\mathcal{S}(c, r)$	Sphere (in the plane: circle) with centre c and radius r .
$\text{bis}(X, Y)$	Bisector of two points: all points equidistant from X and Y .
$\sigma(\mathcal{S}; \eta)$	Selection policy choosing one element of a candidate set $\mathcal{S}$.
$\kappa_2(M)$	Spectral condition number of a matrix M .
$\varepsilon_{\text{mach}}$	Machine epsilon of IEEE double precision, $2^{-52} \approx 2.2 \times 10^{-16}$.

1 Introduction

Computational geometry often begins with a deceptively simple instruction: construct an object from other objects. Draw the circle through three points; drop a perpendicular; find every isosceles triangle whose apex lies on a given line; place a robot joint so that two rigid links reach a target. In an affine coordinate implementation, each problem appears to demand its own collection of formulas and special cases. Parallel lines do not intersect. A circle–circle intersection requires a discriminant. A sphere–sphere intersection is a circle, whereas three spheres typically meet in a pair of points. A robot arm has more than one valid elbow configuration. These facts are geometrically related, but the relationship is easily lost in procedural code.

The source material of this paper contains two particularly attractive ways of recovering that relationship. The first uses homogeneous point and line vectors together with cross products. Parallel lines meet at an ideal point, so the same intersection operation applies to finite and infinite configurations. The second uses conformal geometric algebra, where spheres, planes, circles, and point pairs are represented as blades and are combined through products and duality. In both settings a construction is not merely a list of scalar equations; it is a composition of geometric objects whose algebraic types record what has been constructed.

This shared structure motivates the following research question.

Main research question. How can homogeneous coordinates and conformal geometric algebra be organised as two layers of one computational construction framework, and what do their singular cases reveal about the difference between algebraic elegance and numerical robustness?

Four subordinate questions guide the study:

- RQ1.** Which operations are genuinely common to the planar and the spatial exercises?
- RQ2.** How should multiple algebraic solutions be separated from the policy used to select one physical solution?
- RQ3.** Do the implementations satisfy their geometric constraints, and where does floating-point sensitivity become visible?
- RQ4.** Are the submitted programs correct, and which of their failure modes are geometric and which are artefacts of the chosen formulation?

The contribution is fivefold. First, six exercises are reformulated with a common vocabulary of embedding, join, meet, factorisation, and selection. Second, the role of points at infinity is connected to the role of the null vector e_∞ in CGA without claiming that the two models are identical. Third, branch-selection rules are made explicit and, where possible, derived from a physical criterion instead of a heuristic. Fourth, three exact statements are proved: the number of isosceles apices on a parallel (Proposition 1), the behaviour of the point-pair factorisation used in CLUCalc (Lemma 1), and the geometry of the three-link construction (Proposition 2). Fifth, an independent numerical study verifies the constraints, quantifies two geometric degeneracies, exposes a representation singularity, and re-executes all CLUCalc constructions in an independent implementation of $\mathcal{G}_{4,1}$.

This focus is consistent with the applied-mathematics perspective: the paper does not stop at symbolic derivation, but asks how the formulation behaves in finite precision

and how a program should report a singular configuration. The remainder is organised as follows. Section 2 reviews the two lifts and related work. Section 3 states the scope of the source tasks. Sections 4 and 5 develop the two layers, and Section 6 gives the unified algorithmic pipeline. Sections 7 and 8 describe the experiments and their results, and Section 9 discusses limitations and engineering consequences.

2 Mathematical Background and Related Work

2.1 Why lift a geometric problem?

A coordinate lift represents an object in a space of higher dimension in order to make an important operation simpler. Homogeneous coordinates replace an affine point (x, y) by the equivalence class of triples (x, y, w) with $w \neq 0$; the extra coordinate allows translations and projective transformations to be written as linear maps. Conformal coordinates lift $\mathbb{R}^3$ into a five-dimensional space with an indefinite metric; the extra dimensions linearise a much richer set of objects and transformations. The cost is redundancy and scale freedom. The benefit is a smaller and more regular set of construction operators.

Projective geometry formalises incidence independently of a privileged affine chart. Classical treatments emphasise that parallelism is an affine notion: in the projective completion, parallel lines share an ideal point (Coxeter, 1987). This representation is central to computer vision because points, lines, cameras, and homographies all admit compact homogeneous formulas (Hartley and Zisserman, 2004). It is important, however, not to overstate the invariance. The join and meet are projective; Euclidean perpendicularity and circle radii are not. The planar programs studied here therefore use a projective carrier for incidence together with explicit Euclidean metric operations.

Geometric algebra extends Grassmann’s outer product (Grassmann, 1844) with an inner product and a geometric product. The result represents subspaces, metric relations, and transformations in one algebraic setting (Hestenes and Sobczyk, 1984; Dorst et al., 2007). The conformal model embeds Euclidean geometry so that spheres and planes become linear constraints on embedded points. It is a generalised homogeneous coordinate model, in which lines can be interpreted as circles through infinity (Li et al., 2001). Implementations such as CLUCalc (Perwass, 2009), Gaalop (Hildenbrand, 2013), and more recent robotics-oriented libraries (Löw et al., 2023) demonstrate that the representation is practical, although careful type and sparsity handling are needed because a general multivector of $\mathcal{G}_{4,1}$ has 32 coefficients.

2.2 Geometric representations in robot kinematics

Inverse kinematics asks for joint configurations that realise a prescribed end-effector position or pose. Matrix and screw-theoretic formulations provide the standard systematic language for rigid motion (Murray et al., 1994). A geometric construction offers another perspective: a revolute joint is constrained to spheres, circles, or planes, and inverse kinematics becomes the intersection of these primitives. In CGA these objects inhabit one algebra, so a positional subproblem can be expressed as a sequence of meets. Hildenbrand et al. (2008) formulate the inverse kinematics of a human-arm-like chain in this way, Kleppe and Egeland (2016) solve six-axis industrial manipulators, and Zaplana

et al. (2022) derive closed-form solutions for serial robots with a spherical wrist and extend them to redundant manipulators. The present paper uses only low-dimensional arms, but they expose the essential phenomena: reachability, two branches, tangency, and singularity.

2.3 Algebraic correctness and numerical robustness

An exact representation does not make floating-point arithmetic exact. Orientation tests, intersections, and normalisations can change behaviour when the defining configuration is close to degenerate. Robust computational geometry therefore distinguishes the mathematical predicate from its numerical evaluation; adaptive precision is one established response (Shewchuk, 1997). In numerical linear algebra, the condition number quantifies how strongly input errors may be amplified by a problem, whereas the stability of an algorithm describes how much additional error a particular formulation introduces (Higham, 2002). This distinction — problem versus formulation — is central to Section 6.4: a construction may be unstable at a configuration where the underlying problem is perfectly well-conditioned.

3 Source Constructions and Study Design

The study begins from two sets of course programs, summarised in Table 1. The first set (Aufgaben 1, 2, and 4 of the first practice sheet) is written in Python with the course library `libcfcg`, whose point and line objects support homogeneous cross products. The second set (Hausaufgaben 1–3) is written in CLUCalc (Perwass, 2009) in the outer-product null-space mode `N3_OPNS` and is accompanied by a short explanatory report. The goal is not to reproduce that report page by page, but to extract the strongest idea in each task, to state it precisely, and to reorganise the tasks into one scientific narrative.

Table 1: Source constructions and the geometric idea retained in this study.

Source	Model	Construction	Retained idea
Aufgabe 1 (Python)	$\mathbb{P}^2$ carrier	Circumcircle	Two bisectors are lines; their meet is the centre.
Aufgabe 2 (Python)	$\mathbb{P}^2$ carrier	Perpendicular foot	Parallel perpendiculars meet at an ideal point, which is reused.
Aufgabe 4 (Python)	$\mathbb{P}^2 + \text{circles}$	Isosceles apices	One linear and two circular loci yield up to five apices.
Hausaufgabe 1 (CLUCalc)	CGA $\mathcal{G}_{4,1}$	Tripod	Three spheres meet in a point pair; a fourth support follows from a circle–plane meet.
Hausaufgabe 2 (CLUCalc)	CGA $\mathcal{G}_{4,1}$	Two-link arm	Two link spheres and a helper plane produce two elbows.
Hausaufgabe 3 (CLUCalc)	CGA $\mathcal{G}_{4,1}$	Three-link arm	Two bisector planes produce an isosceles trapezoid of three unit links.

Four design decisions turn these exercises into a coherent study.

First, geometric creation and application policy are separated. An intersection routine returns every mathematically valid branch. A later rule chooses, for example, the point with greater height. This prevents a semantic decision from being mistaken for an algebraic identity.

Second, all outputs are validated in Euclidean coordinates. Even if an object was produced by a projective cross product or a conformal meet, its final physical constraints are distances or incidences. These can be checked independently of the construction path.

Third, degeneracy is treated as a result. Collinear points, tangent circles, coincident joints, and unreachable targets are not exceptions to hide. They indicate that the geometric solution set has changed dimension or multiplicity — or that the chosen construction has lost information. The implementation should classify them before normalising a near-zero homogeneous vector or taking the square root of a negative discriminant.

Fourth, the source programs are themselves objects of study. Each CLUCalc construction is transcribed line by line into an independent NumPy implementation of $\mathcal{G}_{4,1}$ and executed on thousands of configurations. The planar projective constructions are reimplemented independently in `src/geometry.py` and checked through residual and covariance tests. The course-library scripts in `projective_geometry/` require the external `libcfcg` teaching library and are therefore not executed by the automated test workflow. The numerical layer is compared with closed-form Euclidean and projective solutions, turning the exercises into testable specifications and revealing the defects documented in Section 8.4.

4 Projective Construction Layer

4.1 Points, lines, joins, and meets

A homogeneous point of the projective plane is a non-zero vector

$$p = (x, y, w)^T \in \mathbb{R}^3 \setminus \{0\}, \quad p \sim \lambda p, \quad \lambda \neq 0. \quad (1)$$

If $w \neq 0$, the affine representative is $(x/w, y/w)$. If $w = 0$, the point is ideal and records a direction. A line $\ell = (a, b, c)^T$ represents the set of points satisfying

$$\ell^T p = ax + by + cw = 0. \quad (2)$$

The join of two points and the meet of two lines use the same cross product,

$$\ell = p \times q, \quad s = \ell \times m, \quad (3)$$

because $p \times q$ is orthogonal to p and q and therefore satisfies Equation (2) for both. The result is defined up to scale; it vanishes if and only if the two arguments represent the same projective element. These equations are the key reason why the source programs remain compact: the data type changes its geometric meaning, but the low-level operation does not.

Under a non-singular homography H , points and lines transform differently,

$$p' = Hp, \quad \ell' = H^{-T}\ell. \quad (4)$$

Incidence is preserved because $\ell'^T p' = \ell^T H^{-1} H p = \ell^T p$. Moreover, $(Hp) \times (Hq) = \det(H) H^{-T} (p \times q)$, so joins and meets are covariant up to scale. This covariance is tested in the first numerical experiment.

4.2 Circumcentre from two perpendicular bisectors

Let $P_0, P_1, P_2 \in \mathbb{R}^2$ be non-collinear. The midpoint of the edge $P_i P_j$ is $M_{ij} = (P_i + P_j)/2$, and its direction is $d_{ij} = P_j - P_i$. A Euclidean perpendicular direction is

$$d_{ij}^\perp = (-d_{ij,y}, d_{ij,x}). \quad (5)$$

Embedded as the ideal point $v_{ij}^\perp = (d_{ij,x}^\perp, d_{ij,y}^\perp, 0)^\top$, it defines the perpendicular bisector and the centre

$$b_{ij} = m_{ij} \times v_{ij}^\perp, \quad c = b_{01} \times b_{12}, \quad (6)$$

where m_{ij} is the homogeneous embedding of M_{ij} . The radius is $r = \|C - P_0\|$ after dehomogenising c to the affine point C .

The construction cleanly separates two kinds of structure. Computing $d_{ij}^\perp$ uses the Euclidean metric and orientation. Joining M_{ij} to the ideal direction and intersecting the two bisectors are projective incidence operations. Describing the whole procedure as “projectively invariant” would therefore be incorrect, but homogeneous coordinates still remove the distinction between a line defined by two finite points and a line defined by a finite point and a direction.

The construction fails exactly when

$$\Delta = \det \begin{pmatrix} x_0 & y_0 & 1 \\ x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \end{pmatrix} = 0. \quad (7)$$

Then the points are collinear, the bisectors are parallel, and their meet is an ideal point rather than a finite circle centre. Near $\Delta = 0$ the centre is finite but poorly conditioned (Section 7.5). Since Δ scales quadratically with the coordinates, a meaningful collinearity test must be relative, for instance

$$|\Delta| \leq \tau \|P_1 - P_0\| \|P_2 - P_0\|, \quad (8)$$

whereas the submitted program compares $|\Delta|$ with the absolute threshold 10^{-3} , a test whose outcome depends on the unit of the input file.

4.3 Perpendicular transport through an ideal point

Suppose $L = p_0 \times p_1 = (a, b, c)^\top$. Its Euclidean normal direction is (a, b) , so the corresponding ideal point is

$$I_\perp = (a, b, 0)^\top. \quad (9)$$

The perpendiculars through P_0 and P_1 are $n_0 = p_0 \times I_\perp$ and $n_1 = p_1 \times I_\perp$. Their meet $S_1 = n_0 \times n_1$ is again $I_\perp$ up to scale, because parallel lines meet at the ideal point encoding their common direction. The perpendicular through any third point P_2 and its foot are therefore

$$n_2 = S_1 \times p_2, \quad f = n_2 \times L. \quad (10)$$

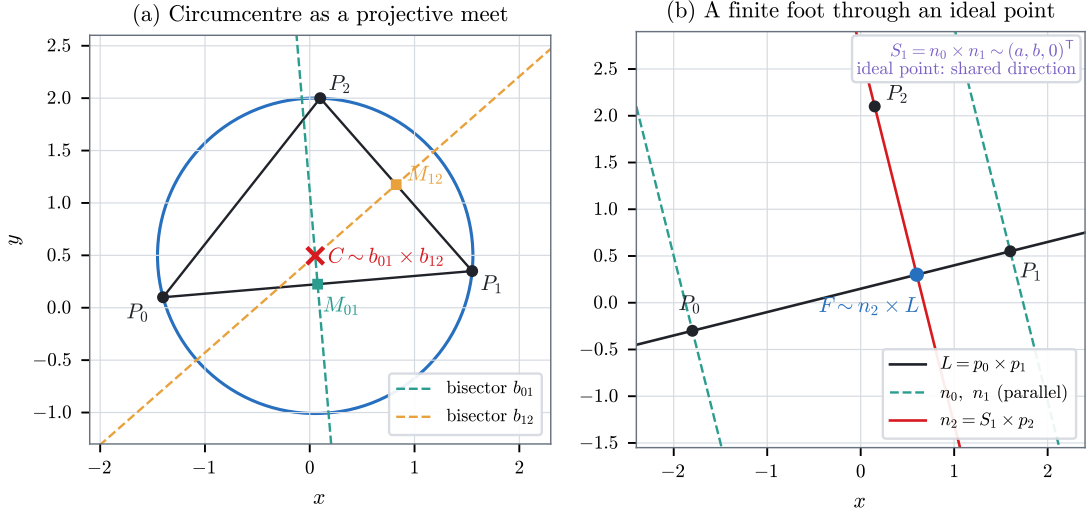


Figure 1: Two uses of homogeneous incidence. (a) Euclidean perpendicular directions define projective bisector lines whose meet is the circumcentre. (b) The perpendiculars n_0, n_1 through P_0 and P_1 are parallel and meet at the ideal point S_1 . The join of S_1 with P_2 is the perpendicular through P_2 , whose finite intersection with the base line is the foot F . The symbol $\sim$ denotes equality up to a non-zero factor.

The source exercise deliberately constructs S_1 as the intersection of the first two perpendiculars before reusing it. Algebraically, one could use the direction $I_\perp$ directly. Pedagogically, the longer route is valuable because it makes the projective closure visible: the “missing” intersection of two parallel lines is a valid object rather than an exceptional return value. A correct program should nevertheless verify that S_1 is ideal with a scale-aware test, see Equation (31) below. A short derivation is given in Appendix A.1.

4.4 Isosceles apices as intersections of loci

Let $P_1 \neq P_2$ be a fixed base of length $d = \|P_2 - P_1\|$, and let $L_\parallel$ be the line through P_0 parallel to P_1P_2 . An apex Q on $L_\parallel$ forms an isosceles triangle P_1P_2Q whenever at least one of the following conditions holds:

$$\|Q - P_1\| = \|Q - P_2\|, \quad (11)$$

$$\|Q - P_1\| = d, \quad (12)$$

$$\|Q - P_2\| = d. \quad (13)$$

The first locus is the perpendicular bisector of P_1P_2 ; the second and third are the circles $\mathcal{S}(P_1, d)$ and $\mathcal{S}(P_2, d)$. The number of apices depends only on the ratio of the height h of $L_\parallel$ above the base to the base length.

Proposition 1 (Isosceles apices on a parallel). *Let h be the signed distance of $L_\parallel$ from the line P_1P_2 . If $h = 0$, every candidate triangle is degenerate. Otherwise the number N of distinct apices $Q \in L_\parallel$ for which P_1P_2Q is isosceles is*

$$N = \begin{cases} 5, & |h| < d \text{ and } |h| \neq \frac{\sqrt{3}}{2}d, \\ 3, & |h| = \frac{\sqrt{3}}{2}d \text{ or } |h| = d, \\ 1, & |h| > d. \end{cases} \quad (14)$$

Proof. Choose coordinates with $P_1 = (0,0)$, $P_2 = (d,0)$, and $L_{\parallel} = \{y = h\}$. Equation (11) gives $x = d/2$, i.e. exactly one apex for every h , because the bisector is perpendicular to $L_{\parallel}$. With $s = \sqrt{d^2 - h^2}$, Equation (12) gives $x = \pm s$ and Equation (13) gives $x = d \pm s$: two points each if $|h| < d$, one (tangency) if $|h| = d$, and none if $|h| > d$. A point of $\mathcal{S}(P_1, d)$ coincides with a point of $\mathcal{S}(P_2, d)$ only if $s = d - s$, i.e. $s = d/2$, which is equivalent to $|h| = \frac{\sqrt{3}}{2}d$; the common point is then $x = d/2$, the bisector apex, so three candidates merge into one (the equilateral triangle) and $5 - 2 = 3$ distinct apices remain. For $|h| = d$ the points $x = 0$, $x = d/2$, $x = d$ are distinct. No other coincidences are possible, and $h \neq 0$ guarantees non-degenerate triangles. $\square$

The construction is more informative than directly solving a quadratic because it retains the reason for each solution family, see Figure 2. The submitted program computes the circle intersections correctly but reports neither tangency nor coincidence, so in the equilateral configuration the same apex is drawn three times; the revised program in Appendix B labels every apex with its locus and merges coincident ones.

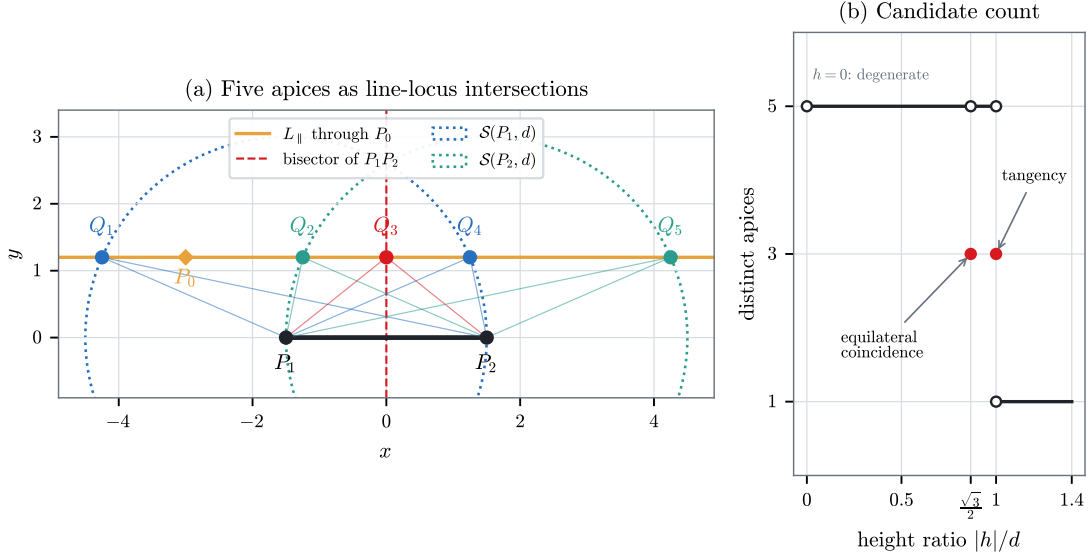


Figure 2: (a) The five generic isosceles apices on a line parallel to the fixed base: Q_3 lies on the perpendicular bisector, Q_1, Q_4 on $\mathcal{S}(P_1, d)$, and Q_2, Q_5 on $\mathcal{S}(P_2, d)$. (b) Number of distinct apices as a function of $|h|/d$ (Proposition 1); open circles mark excluded values, filled dots the isolated values at the equilateral coincidence $|h|/d = \sqrt{3}/2$ and at tangency $|h|/d = 1$; $h = 0$ is degenerate.

5 Conformal Geometric Algebra Layer

5.1 Embedding Euclidean points

The conformal model of $\mathbb{R}^3$ uses the geometric algebra $\mathcal{G}_{4,1}$ generated by e_1, e_2, e_3, e_+, e_- with $e_i^2 = e_+^2 = 1$ and $e_-^2 = -1$. The two null vectors

$$e_{\infty} = e_- + e_+, \quad e_0 = \frac{1}{2}(e_- - e_+), \quad e_{\infty}^2 = e_0^2 = 0, \quad e_0 \cdot e_{\infty} = -1, \quad (15)$$

represent the point at infinity and the origin (CLUCalc: `einf`, `e0`). A Euclidean point $x \in \mathbb{R}^3$ is embedded as the null vector

$$X = \mathcal{C}(x) = e_0 + x + \frac{1}{2} \|x\|^2 e_\infty. \quad (16)$$

This normalisation gives $X^2 = 0$ and $X \cdot e_\infty = -1$; for a scaled point λX the *weight* is $-\lambda X \cdot e_\infty = \lambda$. Most importantly, Euclidean distance is recovered through the inner product,

$$X \cdot Y = -\frac{1}{2} \|x - y\|^2. \quad (17)$$

Thus metric information that was external to the projective carrier becomes an intrinsic bilinear relation of the conformal space. For instance, the vector $X - Y$ satisfies $Z \cdot (X - Y) = \frac{1}{2} (\|z - y\|^2 - \|z - x\|^2)$ for every point Z , so it represents the bisector plane $\text{bis}(x, y)$ in IPNS.

There are two common representation conventions. In OPNS an object is the outer product of points lying on it; in IPNS the dual object acts as a linear constraint. With the dual $A^* = AI^{-1}$, $I = e_1 e_2 e_3 e_+ e_-$, the meet of two OPNS objects can be written as

$$A \cap B = (A^* \wedge B^*)^*. \quad (18)$$

Signs and scales depend on conventions, but the geometric type of the result does not. The CLUCalc expression `*(A ^ B)` is the software-level form of Equation (18). Table 2 lists the primitives used in the three CLUCalc programs.

Table 2: Conformal primitives used in the CLUCalc programs. C is the normalised conformal point of a centre c , r a radius, n a unit normal, and δ the signed distance of a plane from the origin.

Primitive	Representation	Geometric meaning
Point	$X = e_0 + x + \frac{1}{2} \ x\ ^2 e_\infty$	Null vector for $x \in \mathbb{R}^3$.
Sphere (IPNS)	$s = C - \frac{1}{2} r^2 e_\infty$	$X \cdot s = 0$ iff $\ x - c\ = r$.
Plane (IPNS)	$\pi = n + \delta e_\infty$	$X \cdot \pi = 0$ iff $n \cdot x = \delta$.
Bisector (IPNS)	$m = X_1 - X_2$	Points equidistant from x_1 and x_2 .
Plane (OPNS)	$X_1 \wedge X_2 \wedge X_3 \wedge e_\infty$	Plane through three points (Bodenplatte).
Line (OPNS)	$X_1 \wedge X_2 \wedge e_\infty$	Circle through two points and infinity.
Circle (OPNS)	$X_1 \wedge X_2 \wedge X_3$	Circle through three points.
Point pair (OPNS)	$P = X_+ \wedge X_-$	Two real, one double, or two imaginary points.

The point-pair type is especially useful. A sphere–sphere–plane intersection does not force a program to choose a solution; it returns one object encoding both. Its square classifies the pair: $P \cdot P > 0$ for two real points, $P \cdot P = 0$ for a tangent (double) point, and $P \cdot P < 0$ for an imaginary pair. The CLUCalc programs factorise a real pair with the formula

$$P_\pm = \frac{\pm \sqrt{P \cdot P} + P}{e_\infty \cdot P}, \quad (19)$$

where the division is multiplication by the inverse vector $(e_\infty \cdot P)^{-1}$, and then “normalise” by the *multiplication* $N_\pm = P_\pm (-P_\pm \cdot e_\infty)$. The following lemma, proved in Appendix A.4, explains why this works and what it hides.

Lemma 1 (Point-pair factorisation). *Let $P = \lambda X \wedge Y$ with normalised points $X \neq Y$ and $\lambda \in \mathbb{R} \setminus \{0\}$. Then $P \cdot P = \lambda^2(X \cdot Y)^2 > 0$ and*

$$\{P_+, P_-\} = \{-X, -Y\}, \quad P_+ = \begin{cases} -Y, & \lambda > 0, \\ -X, & \lambda < 0. \end{cases} \quad (20)$$

Two consequences matter for the audit. First, both extracted points have weight exactly -1 , so the multiplication by $-P_\pm \cdot e_\infty = -1$ coincides with the correct division by the weight — but only because the weight has modulus one. Robust code divides. Second, *which* point is returned first is not a geometric property: it flips with the sign of λ , i.e. with the order of the arguments and with the sign conventions of dual and meet. A program that relies on “the first point” instead of an explicit selection rule inherits a hidden convention dependence (Sections 5.4 and 8.4).

5.2 Tripod from a three-sphere meet

Let the fixed base points be A, B, C and let the leg lengths be r_A, r_B, r_C . The apex S must satisfy

$$\|S - A\| = r_A, \quad \|S - B\| = r_B, \quad \|S - C\| = r_C. \quad (21)$$

In CGA, the IPNS sphere vectors s_A, s_B, s_C of Table 2 meet in a point pair,

$$P_S = (s_A \wedge s_B \wedge s_C)^* \simeq S_+ \wedge S_-, \quad (22)$$

which is the CLUCalc expression `*(Kugel11 ^ Kugel12 ^ Kugel13)` for the OPNS spheres $\text{Kugel}_i \simeq s_i^*$, up to sign and scale. When the centres lie in a common ground plane, the two solutions are mirror images across that plane, since the reflection preserves all three distances. Selecting the point with positive height is a physical policy, not part of the meet itself.

The source task adds a fourth leg of prescribed length r_4 . Its possible feet form the circle in which the sphere around the selected apex cuts the ground plane. The helper plane $\tau = A \wedge S_+ \wedge S_- \wedge e_\infty$ — the vertical plane through A and the apex, vertical because $S_+ S_-$ is perpendicular to the ground — cuts this circle in two candidate feet, and the source program chooses the one farther from A :

$$K_4 = \mathcal{S}(S_+, r_4) \cap \pi_{\text{ground}}, \quad \{Q_{\text{out}}, Q_{\text{in}}\} = K_4 \cap \tau, \quad (23)$$

$$Q_{\text{out}} = \arg \max_{Q \in K_4 \cap \tau} \|Q - A\|.$$

The report motivates the fourth leg by the instability of the tripod. This claim can be made precise with the classical criterion of static stability.

Definition 1 (Stability margin). Let a structure rest on ground contacts $G_1, \dots, G_k$ and carry its load at a point S . With proj the orthogonal projection onto the ground plane and $\Omega = \text{conv}\{G_1, \dots, G_k\}$ the support polygon, the stability margin is the signed distance

$$\mu = \begin{cases} \text{dist}(\text{proj } S, \partial\Omega), & \text{proj } S \in \Omega, \\ -\text{dist}(\text{proj } S, \Omega), & \text{proj } S \notin \Omega. \end{cases} \quad (24)$$

The structure is statically stable if and only if $\mu > 0$.

With this definition the choice of the fourth foot becomes a policy with a physical meaning, $\sigma_{\text{stab}} = \arg \max_Q \mu(\text{conv}\{A, B, C, Q\}, S_+)$, which can be compared with the distance heuristic of Equation (23) (Section 8.2). The support polygon is computed with a standard convex-hull algorithm (de Berg et al., 2008). The whole construction is a good example of the full pipeline: the algebra produces a circle and a point pair; the application attaches a stability criterion to the pair.

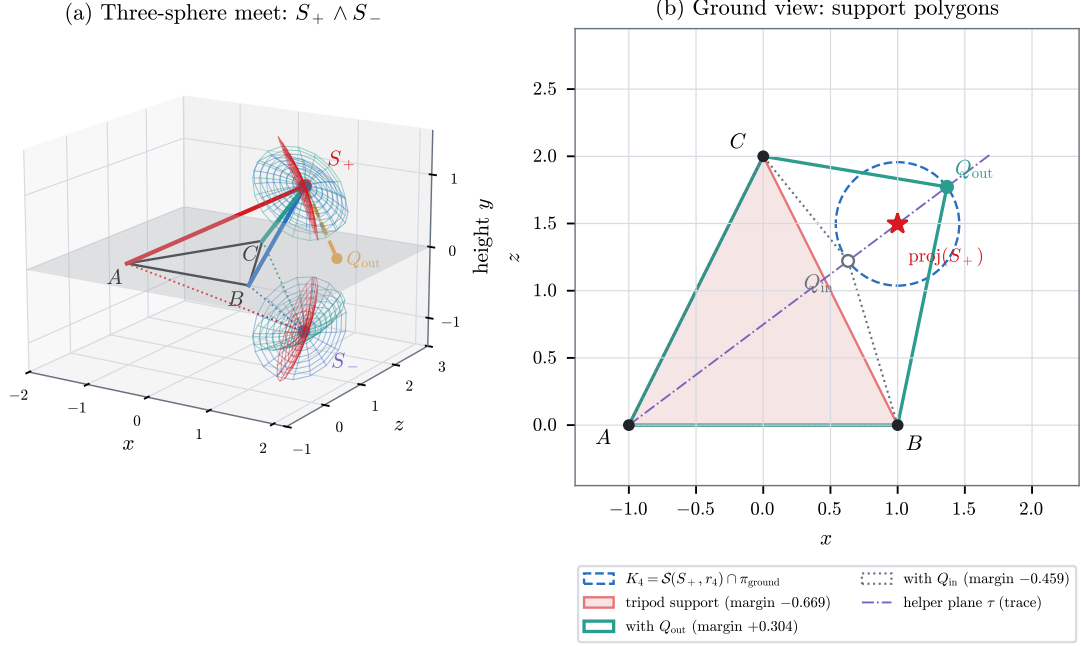


Figure 3: Tripod construction. (a) Patches of the three leg spheres around their two common points: the meet is the point pair $S_+ \wedge S_-$, whose points are mirror images in the ground plane; the upper point S_+ is selected and the fourth leg ends at Q_{out} . (b) Ground view: the projection of S_+ lies outside the support triangle ABC (negative margin); of the two feet on the circle K_4 cut out by the helper plane τ , only Q_{out} yields a support polygon containing the projected apex.

5.3 Two-link inverse kinematics

Consider a shoulder at O , a target T , and link lengths ℓ_1, ℓ_2 . The elbow E must lie on both link spheres:

$$\|E - O\| = \ell_1, \quad \|E - T\| = \ell_2. \quad (25)$$

In three dimensions their intersection is generally a circle. A helper plane through O and T fixes the intended motion plane, and the circle-plane meet is a point pair $E_+ \wedge E_-$. This mirrors the CLUCalc source directly, which uses the vertical plane through O , the point $Z = (0, 1, 0)$, and T ($\mathbf{U}^Z \mathbf{A}^{\text{einf}}$). That plane is undefined when T lies on the vertical axis through O , a degeneracy that the revised program handles with a fallback direction.

In a two-dimensional chart of the helper plane, let $d = \|T - O\|$, $u = (T - O)/d$, and let $u_\perp$ be a unit vector perpendicular to u in that plane. Then

$$a = \frac{\ell_1^2 - \ell_2^2 + d^2}{2d}, \quad h = \sqrt{\ell_1^2 - a^2}, \quad E_\pm = O + au \pm hu_\perp. \quad (26)$$

Real solutions exist exactly when

$$|\ell_1 - \ell_2| \leq d \leq \ell_1 + \ell_2. \quad (27)$$

Strict inequalities give two branches, equality gives a tangent (fully stretched or folded) configuration, and violation gives no real elbow. A rule such as “choose the greater vertical coordinate” should only be applied after this classification; the submitted program lets its sliders reach targets with $d > 2$ and passes the resulting imaginary pair to the square root.

5.4 Three-link arm: an isosceles trapezoid

The third program moves an end point T with three unit links $O \rightarrow E_1 \rightarrow E_2 \rightarrow T$. Its construction is less obvious than its report suggests. It first intersects the line OT with the sphere $\mathcal{S}(T, 1)$, which gives the two auxiliary points $W_{\pm} = T \pm u$, $u = (T - O)/d$. It then intersects the bisector plane of O and the first auxiliary point with $\mathcal{S}(O, 1)$, the bisector plane of O and the second auxiliary point with $\mathcal{S}(T, 1)$, and cuts both circles with the helper plane Π . The following proposition identifies the result.

Proposition 2 (Three-link construction). *Let $0 < d = \|T - O\| \leq 3$, let $u_{\perp}$ be the upward unit vector perpendicular to u in Π , and let E_1, E_2 be the upper points of*

$$\text{bis}(O, W_-) \cap \mathcal{S}(O, 1) \cap \Pi \quad \text{and} \quad \text{bis}(O, W_+) \cap \mathcal{S}(T, 1) \cap \Pi. \quad (28)$$

Then, with $h = \sqrt{1 - (d - 1)^2/4}$,

$$E_1 = O + \frac{d-1}{2}u + hu_{\perp}, \quad E_2 = O + \frac{d+1}{2}u + hu_{\perp} = E_1 + u. \quad (29)$$

Hence $\|E_1 - O\| = \|E_2 - E_1\| = \|T - E_2\| = 1$: the chain OE_1E_2T is an isosceles trapezoid with $E_1E_2 \parallel OT$, symmetric with respect to $\text{bis}(O, T)$, and E_1 is the mirror image of E_2 in $\text{bis}(O, T)$. The first set in Equation (28) is undefined at $d = 1$, where $W_- = O$.

The proof (Appendix A.5) reduces to the observation that $\text{bis}(O, wu)$ is the plane $\{x : (x - O) \cdot u = w/2\}$. Three consequences follow.

The program is correct, but for a non-obvious reason. The auxiliary points $W_{\pm}$ are not wrists of the arm; they only position the two bisector planes. The arm itself is the trapezoid of Equation (29), visualised in Figure 4(b).

Correctness depends on the order of the extracted pair. If the factorisation returns $W_{\pm}$ in the opposite order — which, by Lemma 1, is a matter of sign conventions — the program intersects $\text{bis}(O, W_+)$ with $\mathcal{S}(O, 1)$ instead. The resulting crossed chain exists only for $d \leq 1$, so a fraction $1 - 1/27 \approx 96.3\%$ of the reachable ball $\|T - O\| \leq 3$ would be lost.

The point $d = 1$ is a representation singularity. At $d = 1$ the arm is regular ($h = 1$), yet $W_- = O$ and the IPNS bisector $O - W_-$ is the zero vector. Near $d = 1$ the vector $O - W_-$ is obtained by cancellation, and rounding errors of order $\varepsilon_{\text{mach}}$ tilt the bisector plane by an angle of order $\varepsilon_{\text{mach}}/|d - 1|$ (Appendix A.6). The revised construction avoids W_- altogether: W_+ is selected explicitly as the auxiliary point farther from O

(it satisfies $\|W_+ - O\| = d + 1 \geq 1$), E_2 is obtained from one circle-plane meet, and E_1 by the reflection versor

$$E_1 = -m E_2 m^{-1}, \quad m = O - T, \quad (30)$$

where m is the IPNS bisector plane of O and T . Reflections preserve the weight of a conformal point, so E_1 is again normalised. The revised construction is exact for all $0 < d \leq 3$; its only remaining singularities are geometric: the direction u is undefined at $d = 0$, and the two branches merge at full extension $d = 3$.

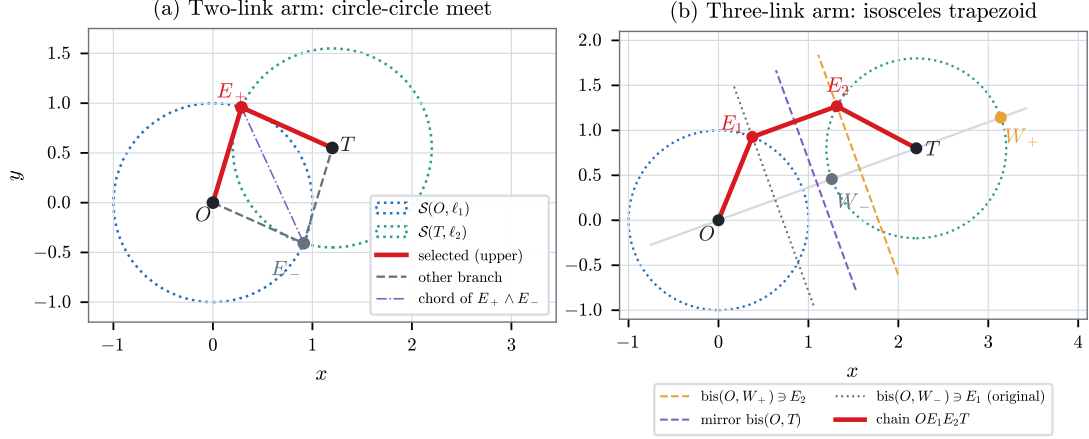


Figure 4: Kinematic constructions in the helper plane. (a) Two unit circles meet at the two elbows $E_{\pm}$; the upper one is selected after both have been generated. (b) The three-link program: E_2 lies on $\mathcal{S}(T, 1)$ and on the bisector of O and $W_+ = T + u$; E_1 is its mirror image in $\text{bis}(O, T)$, so OE_1E_2T is an isosceles trapezoid with three unit sides (Proposition 2). The dotted plane $\text{bis}(O, W_-)$ used by the original program passes through the same E_1 ; it degenerates when $d \rightarrow 1$. Target distance $d \approx 2.34$.

6 A Unified Construction Pipeline

The planar and conformal tasks differ in dimension and metric content, but their programs can be organised by the same six-stage pipeline shown in Table 3. This organisation is the main synthesis of the paper.

Table 3: Common pipeline for the two representation layers.

Stage	Question	Projective layer	Conformal layer
1	Encode	Lift (x, y) to $(x, y, 1)$; directions use $w = 0$.	Lift x to the null point $X = \mathcal{C}(x)$.
2	Construct	Form lines by point–point or point–direction joins.	Form spheres, planes, bisectors, lines, and circles as blades.
3	Intersect	Cross product of two line vectors.	Duality and outer product, Equation (18).
4	Factor	Dehomogenise a finite point or retain an ideal point.	Classify by $P \cdot P$; extract both points (Lemma 1); divide by the weight.
5	Select	Apply domain rules only if several objects remain.	Explicit policy σ : height, stability margin, continuity.
6	Validate	Incidence and Euclidean residuals.	Nullness, distance constraints, reality.

6.1 Type before coordinates

The first safety principle is to inspect the type and scale of an algebraic result before converting it into ordinary coordinates. For a projective point $p = (x, y, w)^\top$ the program should compare $|w|$ with a scale-aware tolerance,

$$|w| \leq \tau \|p\|_2. \quad (31)$$

If the condition holds, dividing by w is unsafe and the result should be reported as ideal or near-ideal. For a conformal point pair, the analogous step is to classify the blade as real, tangent, or imaginary by the sign of $P \cdot P$ relative to its scale before extraction.

This is more than defensive programming. The classification carries geometric meaning. A circumcentre at infinity states that the defining points are collinear. A tangent elbow pair states that the arm is fully stretched or folded. An imaginary point pair states that the requested distances are incompatible.

6.2 Branch policy as an explicit function

Let an intersection return a finite candidate set $\mathcal{S} = \{q_1, \dots, q_k\}$. A selection policy is a separate function

$$\sigma(\mathcal{S}; \eta) \longrightarrow q_i, \quad (32)$$

where η contains application context. Examples are height, distance from a previous joint, support-polygon margin, or temporal continuity. Written this way, the same construction can serve different applications without changes to the intersection code.

The “upper point” policy of the source tasks is $\sigma_{\text{up}}(\mathcal{S}) = \arg \max_{q \in \mathcal{S}} e_2^\top q$; it is deterministic except when two candidates have equal height, which happens exactly at tangency. The stability policy σ_{stab} of Section 5.2 replaces a heuristic by a physical criterion. For an animated arm, a continuity policy is usually better:

$$\sigma_{\text{cont}}(\mathcal{S}; q_{\text{prev}}) = \arg \min_{q \in \mathcal{S}} \|q - q_{\text{prev}}\|. \quad (33)$$

It prevents a small target movement from causing an arbitrary elbow flip. In every case the policy acts on *named* candidates. Lemma 1 shows why the order in which an algebra happens to return them must never act as an implicit policy.

6.3 Residuals as contracts

Every stage should expose a postcondition. For projective incidence, a scale-invariant residual is

$$\rho_{\text{inc}}(\ell, p) = \frac{|\ell^\top p|}{\|\ell\|_2 \|p\|_2}. \quad (34)$$

For a link or sphere constraint with centre c and radius r we use

$$\rho_{\text{dist}}(x; c, r) = |||x - c||_2 - r|. \quad (35)$$

For a configuration with several constraints, the maximum residual is a simple and interpretable contract. A small residual does not prove that the problem is well-conditioned, but a large residual immediately falsifies the result.

Reference procedure for a geometric intersection task

1. Validate the input scale and reject undefined primitives.
2. Embed the Euclidean inputs in the selected algebraic model.
3. Construct the required loci from named geometric primitives.
4. Compute the meet without selecting or discarding a branch.
5. Classify the result as empty, tangent, multiple, finite, or ideal.
6. Extract every real candidate and apply an explicit policy σ .
7. Evaluate independent residuals and return the solution together with a diagnostic record.

6.4 Geometric versus representation singularities

The examples of Sections 4 and 5 suggest a distinction that is useful when a construction misbehaves.

Definition 2. Let a construction compute the solution $\Phi(x)$ of a geometric problem from input data x through a sequence of intermediate objects. A configuration x^* is a *geometric singularity* if the problem itself degenerates there: the solution set changes cardinality or dimension, or Φ is not locally Lipschitz at x^* . It is a *representation singularity* of the construction if Φ is regular at x^* but an intermediate object of the construction vanishes or becomes undefined.

Collinear triangle vertices and a fully stretched arm are geometric singularities: every formulation must report them, and near them every formulation is sensitive, because the condition number of the problem itself is large. The configuration $d = 1$ of the three-link program is a representation singularity: the trapezoid of Proposition 2 depends smoothly on T there, but the bisector $\text{bis}(O, W_-)$ does not exist. Near a representation singularity the forward error of the construction grows although the problem is well-conditioned. The remedy is a reformulation such as Equation (30), not more precision.

7 Experimental Methodology

7.1 Implementation and reproducibility

The experiments use Python, NumPy double precision, Matplotlib, and the fixed random seed 20260922. The current automated workflow is tested with Python 3.11. The script `src/generate_figures.py` regenerates every reported experimental number, the LaTeX result macros, and all numerical plots from a clean checkout; no reported experimental value is transcribed by hand. The implementation is divided into the following components:

- `src/cga.py`: a self-contained implementation of $\mathcal{G}_{4,1}$ with 32 coefficients per multi-vector, geometric, outer and inner products, inverse, dual $A^* = AI^{-1}$, and CLUCalc counterparts (`VecN3`, `SphereN3`, `meet`, point-pair factorisation);
- `src/geometry.py`: closed-form Euclidean and projective reference solutions of all six tasks, returning structured results (status, candidates, selection, residual);
- `src/cga_constructions.py`: executable NumPy/CGA transcriptions of the three CLUCalc constructions in their original and revised forms;
- `tests/`: automated regression tests for the $G(4,1)$ algebraic identities, conformal embedding and distance relations, point-pair extraction, projective constructions, kinematic constraints, tripod stability, special isosceles-locus heights, the three-link representation singularity, and agreement between the revised CGA constructions and independent reference solutions.

This yields a cross-implementation validation: the constructions are executed in the CGA engine, while their residuals and selected solutions are checked against independent Euclidean and projective formulas.

7.2 Experiment 1: projective covariance

For each of 5,000 trials, four affine points were sampled uniformly from $[-2, 2]^2$ and embedded with $w = 1$. Two lines and their intersection were computed by cross products. A random homography $H = I + 0.22N$, with standard normal entries in N , was accepted when $|\det H| > 0.15$ and $\kappa_2(H) < 50$. Points were transformed by H and lines by $H^{-\top}$. The per-trial error was the maximum of the transformed incidence residuals Equation (34), the angular disagreement between directly recomputed joins and $H^{-\top}\ell$, and the angular disagreement between the directly recomputed meet and Hs . The experiment deliberately does not test the preservation of perpendicularity, because a general homography does not preserve Euclidean angles.

7.3 Experiment 2: constraints of the closed forms

For the two-link arm with unit links, 10,000 planar targets were sampled uniformly by area in the annulus $0.05 \leq d \leq 1.95$, which avoids the origin and the tangent outer boundary. The upper elbow of Equation (26) was selected and the maximum of the two link-length residuals was recorded. For the three-link arm, 10,000 targets were sampled in $0.05 \leq d \leq 2.95$, the trapezoid Equation (29) was evaluated, and the maximum error of the three link lengths was recorded. For Proposition 1, 19,944 random configurations with base length at least 0.2 were drawn from $[-3, 3]^2$; the number of apices found by

the locus construction was compared with Equation (14), including the two exceptional heights, and the isosceles residual was recorded.

7.4 Experiment 3: tripod and support polygon

The source tripod uses

$$\begin{aligned} A &= (-1, 0, 0), & B &= (1, 0, 0), & C &= (0, 0, 2), \\ r_A &= 2.69, & r_B &= 1.8, & r_C &= 1.5, \end{aligned} \quad (36)$$

with the second coordinate as height. Both sphere intersections were obtained by the linear reduction of Appendix A.2, the positive-height branch was selected, and the fourth foot of length 1.1 was computed with Equation (23). The maximum distance residual over the four legs was recorded, and the stability margin Equation (24) was evaluated for the tripod and for both candidate feet, assuming the load at the apex.

7.5 Experiment 4: conditioning near singularities

Three limits were studied. For the circumcentre, the triangle

$$P_0 = (-1, 0), \quad P_1 = (1, 0), \quad P_2 = (0, h) \quad (37)$$

was evaluated for 21 logarithmically spaced altitudes from 10^{-1} to 10^{-11} . The centre solves the linear system $Mc = b$ with $M = 2[P_1 - P_0, P_2 - P_0]^\top$ and $b = (\|P_1\|^2 - \|P_0\|^2, \|P_2\|^2 - \|P_0\|^2)^\top$; for this triangle $\kappa_2(M) = 2.5/h + O(h)$. Each coordinate was perturbed by independent Gaussian noise of standard deviation 10^{-10} in 400 repetitions, and the median relative centre error was recorded.

For the two-link arm, the target distance approached the tangent value $d = 2$. With the elbow height $h = \sqrt{1 - (d/2)^2}$, the branch separation is $2h$, while

$$\left| \frac{dh}{dd} \right| = \frac{d}{4h} \quad (38)$$

diverges as $d \rightarrow 2^-$.

For the three-link arm, targets $T = (1 + \delta)u$ were generated for 20 random directions u and 27 values $\delta \in [10^{-14}, 10^{-1}]$. Both the original and the revised CLUCalc construction were executed in the CGA engine, and the median position error of (E_1, E_2) relative to Equation (29) was recorded.

7.6 Experiment 5: executing the CLUCalc programs

The three CLUCalc programs were transcribed statement by statement, including the point-pair formula Equation (19) and the multiplicative normalisation. For the two-link program, 2,000 targets were sampled uniformly by volume in the shell $0.05 \leq d \leq 1.95$; for the three-link program, 2,000 targets in $0.05 \leq d \leq 2.95$. The elbows of the original and the revised transcription were compared with the closed forms Equations (26) and (29). To expose the ordering dependence of Lemma 1, the three-link program was additionally executed with the two auxiliary points interchanged, which emulates the opposite sign convention. The tripod program was executed once in both versions.

8 Results

8.1 Constraint validation

Table 4 summarises the regular-case validation. All median residuals are at the level of the unit roundoff 1.1×10^{-16} , and every maximum remains below 10^{-13} . The projective experiment has the largest maximum because it composes a matrix inversion, two transformations, and several cross products; the kinematic formulas evaluate only a few normalised vector operations and square roots. The apex count predicted by Proposition 1 agreed with the locus construction in all 19,944 random configurations and at both exceptional heights (0 mismatches).

Table 4: Residual validation in double precision. Projective values combine incidence and covariance errors; the isosceles value is the relative deviation from the nearest isosceles condition; the remaining values are absolute link or sphere-distance errors.

Experiment	Trials	Median residual	Maximum residual
Projective join/meet covariance	5,000	2.24×10^{-16}	3.42×10^{-14}
Isosceles apices (Proposition 1)	19,944	—	5.16×10^{-15}
Two-link distance constraints	10,000	1.11×10^{-16}	4.44×10^{-16}
Three-link distance constraints	10,000	1.11×10^{-16}	4.44×10^{-16}
Tripod and fourth support	1	8.88×10^{-16}	8.88×10^{-16}

The results support the first two research questions. Cross products and conformal meets both serve as a construction grammar, and explicit branch selection does not compromise the constraints when it is applied after the full solution set has been produced. The data do not establish that one algebra is faster than the other; no timing comparison was attempted.

8.2 Numerical tripod and static stability

The selected upper apex and the two candidate feet are

$$\begin{aligned} S_+ &= (0.9990, 0.9995, 1.4970), \\ Q_{\text{out}} &= (1.3667, 0.0000, 1.7724), \quad Q_{\text{in}} = (0.6313, 0.0000, 1.2216). \end{aligned} \tag{39}$$

The reflected apex S_- has the same first and third coordinates and the opposite height. The maximum residual across the three original legs and the added 1.1-unit leg is 8.88×10^{-16} .

The stability margins answer the question that the distance heuristic leaves open. With the load at the apex, the tripod alone has $\mu = -0.669$: the projected apex lies beyond the edge BC of the support triangle, so the structure tips over, which confirms the claim of the original report. Adding Q_{in} still gives $\mu = -0.459$, whereas Q_{out} gives $\mu = +0.304$ (Figure 3b). For this geometry the heuristic “farther from A ” therefore coincides with the stability policy σ_{stab} , and it is the only admissible choice. This agreement is a property of the data, not a theorem: in general, static stability depends on the projection of the centre of mass, and structural stiffness on material and joint properties. The algebra generates the feasible feet; the engineering model decides which one is preferable.

8.3 Behaviour near degeneracy

Table 5 and Figure 5 answer the third research question. As the altitude decreases, the circumcentre system becomes nearly singular: $\kappa_2(M)$ grows like h^{-1} , and the median relative error of the centre follows the same trend until it saturates at order one, when the altitude reaches the noise level. The homogeneous construction still returns a vector, and its incidence residual may remain small, but the dehomogenised centre is no longer a reliable estimate of the unperturbed problem.

Table 5: Circumcentre conditioning for selected triangle altitudes. Each median error is based on 400 perturbations with coordinate noise of standard deviation 10^{-10} .

Altitude h	$\kappa_2(M)$	Median relative centre error
10^{-1}	2.501×10^1	8.722×10^{-10}
10^{-4}	2.500×10^4	8.514×10^{-7}
10^{-7}	2.500×10^7	8.119×10^{-4}
10^{-10}	2.500×10^{10}	6.005×10^{-1}
10^{-11}	2.500×10^{11}	9.576×10^{-1}

The two-link plot shows the same phenomenon in a different language. As the two circles approach tangency, the elbow branches merge and their separation tends to zero, while the derivative of the elbow height with respect to the target distance diverges. A small target perturbation may therefore produce a large change of the elbow height even though both link lengths remain satisfied to machine precision.

The three-link experiment isolates the representation singularity. The error of the original construction grows consistently with $\varepsilon_{\text{mach}}/|d-1|$ over the tested range and reaches 1.09×10^{-9} at $|d-1| = 10^{-7}$, while the revised construction stays below 6.27×10^{-16} for every tested δ (Figure 5c). Since the trapezoid itself is well-conditioned near $d = 1$, this amplification is entirely a property of the formulation.

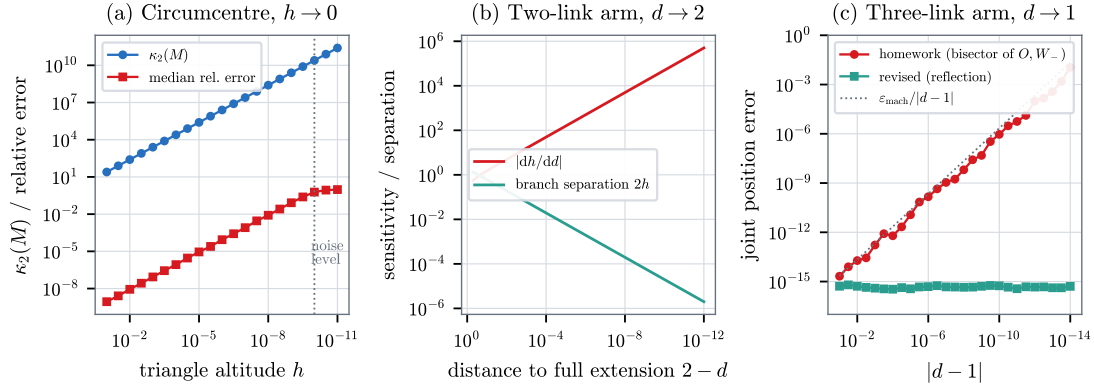


Figure 5: Three limits. (a) Geometric singularity: the circumcentre becomes sensitive as the triangle approaches collinearity. (b) Geometric singularity: the elbow branches merge at full extension while the derivative of their height diverges. (c) Representation singularity: the original three-link construction becomes increasingly inaccurate near $d = 1$, consistently with $\varepsilon_{\text{mach}}/|d-1|$ over the tested range; the reflection-based construction avoids the degenerate intermediate plane and remains near machine precision.

8.4 Audit of the source programs

Table 6 reports the execution of the CLUCalc programs in the CGA engine, and Table 7 lists all observations together with the revisions reproduced in Appendix B.

Table 6: Execution of the CLUCalc programs in the independent CGA engine: maximum Euclidean deviation of the computed joints from the closed-form solutions.

Program	Version	Max. error	Remark
Hausaufgabe 1 (tripod)	original	3.82×10^{-15}	Apex and outer foot correct; the selection is bypassed when drawing.
	revised	3.82×10^{-15}	Selected points used throughout.
Hausaufgabe 2 (two-link)	original	1.14×10^{-15}	2,000 targets; correct off the vertical axis.
	revised	1.14×10^{-15}	Adds reachability test and plane fallback.
Hausaufgabe 3 (three-link)	original	5.54×10^{-13}	Median 1.43×10^{-15} ; the maximum occurs close to $d = 1$ (0 target with error $> 10^{-12}$). With interchanged auxiliary points, 95.5 % of the targets become unreachable (theory for this shell: 96.1 %).
	revised	1.82×10^{-14}	Uniform accuracy; largest errors close to full extension $d \rightarrow 3$.

Table 7: Observations on the submitted programs and the corresponding revisions.

Program	Observation and consequence	Revision
Aufgabe 1	Collinearity test with the absolute threshold $ \Delta < 10^{-3}$; its outcome depends on the unit of the data.	Relative test Equation (8); centre classified by Equation (31); residual reported.
Aufgabe 2	The perpendiculars are called “bisectors”; the ideal point S_1 is printed but not verified.	Naming corrected; scale-aware ideal test; residuals of the foot.
Aufgabe 4	Tangency and coincidences are not reported, so the equilateral apex is drawn three times; P_0 on the base line is not detected; fixed drawing window.	Classification per locus (Proposition 1), merging of coincident apices, degeneracy test, data-driven window.
Hausaufgabe 1	The apex Spitze is selected, but legs and the fourth sphere use N_Pp2 , and the last leg is drawn to N_Pp4 instead of AußenPunkt ; whether this is correct depends on the ordering of the pair (Lemma 1). Duplicated header, non-ASCII identifiers.	Selected points used throughout; header cleaned; ASCII identifiers.
Hausaufgabe 2	Helper plane undefined for targets on the vertical axis; sliders reach $d > 2$ without a reachability test.	Fallback direction; classification by the sign of $P \cdot P$ before factorisation.
Hausaufgabe 3	Correctness relies on the implicit order of N_Pp1 , N_Pp2 ; at $d = 1$ one auxiliary point coincides with the shoulder, so the corresponding bisector plane degenerates.	W_+ chosen by distance; E_1 by the reflection Equation (30).
All CLUCalc	Normalisation by multiplication with $-X \cdot e_\infty$; correct only because the weight is ± 1 .	Division by the weight $-X \cdot e_\infty$.

Two findings deserve emphasis. First, none of the observations changes the geometric idea of a program; they concern selection, classification, and conventions — exactly the stages 4–6 of Table 3 that the exercises leave implicit. Second, the most instructive defect, the representation singularity of Hausaufgabe 3, is easy to miss in interactive use. The singular set $d = 1$ is a sphere in the three-dimensional target parameter space, and the region in which the forward error exceeds 10^{-12} is narrow. A systematic experiment or the geometric proof makes the mechanism much clearer than slider exploration alone.

9 Discussion

9.1 What is genuinely unified?

The most compelling common feature of the tasks is not a particular formula but the conversion of a verbal construction into intersections of named loci. The circumcentre is the intersection of bisectors. The isosceles apices are intersections of a line with a bisector or with circles. The tripod apex is the intersection of three spheres. The elbow is the intersection of two link spheres and a helper plane, and the three-link chain is obtained from two bisector planes. Once the loci are explicit, solution multiplicity and degeneracy follow from their geometry, as Propositions 1 and 2 illustrate.

Homogeneous coordinates and CGA supply compatible algebraic habits: objects are represented up to scale, joins are outer constructions, meets are dual constructions, and ideal elements close the incidence system. CGA goes further by internalising Euclidean distance and representing round primitives directly. The relation should be understood as a progression, not an identity: $\mathbb{P}^2$ is a three-coordinate projective model of planar incidence, whereas $\mathcal{G}_{4,1}$ is a Clifford algebra over the five-dimensional space $\mathbb{R}^{4,1}$ of signature $(4, 1)$.

9.2 The value of visible multiplicity

Ordinary application code often returns one answer because a downstream component expects one. The source tasks are more instructive: they make the pair of sphere intersections or elbow positions visible before choosing. This has three benefits. It prevents silent branch loss. It makes singularity detection natural, because two branches coalesce at tangency. And it supports different policies without changing the geometric solver: an interactive visualisation may choose the upper point, a trajectory the nearest previous joint, and a structural design the foot with the largest stability margin.

The isosceles example generalises the lesson beyond pairs: there are up to five candidates because there are three distinct ways in which a triangle with a fixed base can be isosceles, and Proposition 1 states exactly when they merge. The audit adds a complementary lesson: visible multiplicity is only safe if the candidates are *named* by geometry, not by the order in which the algebra returns them.

9.3 Where the algebra does not help automatically

Five limitations are important.

Numerical scale remains important. Homogeneous objects can be multiplied by arbitrary non-zero scalars. Without normalisation, residual thresholds have no consistent meaning and intermediate coefficients can overflow or underflow. Scale-invariant tests such as Equations (8) and (34) should be preferred.

Degenerate geometry remains degenerate. Representing an ideal circumcentre is mathematically elegant, but a program that needs a finite radius must still stop. Likewise, CGA encodes a tangent point pair compactly, but the branch derivative is still singular.

Constructions can create their own singularities. The three-link program shows that an algebraically correct construction may route the computation through an object that degenerates where the problem does not. Such representation singularities are avoided by choosing intermediate objects that exist throughout the domain, as in Equation (30).

Selection is not solved by representation. CGA returns both elbows, but it cannot decide which one is collision-free or comfortable without additional constraints. The choice belongs to the application model.

A richer algebra has computational overhead. Cross products of three-vectors are inexpensive. General conformal multivector operations evaluate many more coefficients than a specialised Euclidean formula. Sparse typed implementations mitigate this cost (Hildenbrand, 2013; Löw et al., 2023), but the present experiments do not compare execution time or memory, and no performance claim is made.

9.4 Improvements for production implementations

The educational programs can be developed into robust software through several concrete changes:

1. Replace fixed absolute tolerances by scale-aware thresholds.
2. Return structured results containing status, all candidates, the selected candidate, and residuals.
3. Select candidates by explicit, named policies; for time-dependent targets, extend this to continuity-aware branch tracking.
4. Guard orientation, tangency, and reality predicates; adaptive precision remains a useful extension when a discrete decision is numerically ambiguous.
5. Test invariants under rigid motions, scalings, and admissible projective transformations of the incidence subproblems.

The current repository implements the scale-aware tests, structured result records, explicit selection policies, algebraic invariant tests, and randomized covariance/constraint experiments. Continuity-aware trajectory selection and adaptive-precision predicates are left as future work. A possible result record is

$$\text{ConstructionResult} = (\text{status}, \text{candidates}, \text{selected}, \text{residual}, \text{info}), \quad (40)$$

as implemented in `geometry.py`. Such a record makes the diagnostic information part of the public interface rather than console output that a caller may ignore.

9.5 Threats to validity

The study uses synthetic configurations and low-dimensional mechanisms. The residual experiments show that the formulas were implemented consistently, not that they solve all robotics or structural-design problems. Joint limits, obstacles, uncertainty, force transmission, and dynamic effects are absent. The three-link construction selects one symmetric family of configurations and is not a complete parameterisation of the redundancy.

The CLUCalc programs were not run in CLUCalc itself but in an independent implementation of $\mathcal{G}_{4,1}$. This checks the mathematical construction, but software-specific conventions — the sign of the dual, the exact inner-product convention, and rendering behaviour — may differ. For this reason, conclusions that depend on such conventions are stated as dependencies (for instance, on the ordering of a point pair) rather than as failures of the course programs. The scripts in `projective_geometry/` depend on the external `libcfcg` teaching library and are not executed in continuous integration; their geometric content is instead checked by independent NumPy reference implementations. Finally, the perturbation study examines three representative singularities rather than a complete error analysis of all constructions.

10 Conclusion

This paper transformed two sets of geometry exercises into one computational study. The homogeneous layer showed how points, lines, directions, joins, and meets provide a closed language for planar incidence; its most memorable object is the ideal point shared by parallel perpendiculars. The conformal layer extended the same construction-oriented thinking to distance geometry: spheres meet in circles or point pairs, helper planes reduce solution families, and robot joints emerge as intersections of geometric constraints.

The synthesis led to a six-stage pipeline: encode, construct, intersect, factor, select, and validate. Separating the selection stage was decisive. “Upper elbow” and “outer support” are useful policies, but they are not properties of the intersection algebra; for the tripod the latter could be justified by a stability margin, which turned a heuristic into a criterion. Keeping all candidates until selection preserves the geometry and makes tangency and non-uniqueness visible — provided the candidates are identified by geometry rather than by the order in which an algebra returns them.

The numerical experiments confirmed the constraints at machine precision for thousands of regular configurations. They also demonstrated the central caution in two forms. An elegant representation does not cure an ill-conditioned problem: circumcentres remain unstable near collinearity and elbows near tangency. Conversely, an elegant construction can itself be unstable where the problem is harmless, as the representation singularity of the three-link program shows; there, a better formulation — a single reflection — removes the defect completely. The proper conclusion is not to abandon lifted models, but to pair them with classification, scale-aware tolerances, residuals, and explicit diagnostics, and to test constructions against independent formulas.

For future work, the framework can be extended in three directions: a typed implementation that exposes point pairs and ideal points as first-class return types; continuous branch tracking with joint limits and obstacle constraints; and a comparative bench-

mark between specialised Euclidean formulas and sparse CGA code that measures not only speed but also implementation complexity, invariance, and failure behaviour near singularities. Those extensions would preserve what is most valuable in the present tasks: geometry remains visible in the code.

A Selected Derivations

A.1 Why the projective perpendicular construction works

Let $L = (a, b, c)^\top$ be a finite line, i.e. $(a, b) \neq (0, 0)$. Its direction is $(-b, a)$ and its normal direction is (a, b) . The ideal point $I_\perp = (a, b, 0)^\top$ therefore encodes the direction of every line perpendicular to L . For a finite point p , the join $n = p \times I_\perp$ is incident with p and with that ideal point; hence it is precisely the perpendicular through p . The foot $f = (p \times I_\perp) \times L$ is incident with both lines, and for $p = (x, y, 1)^\top$ its third coordinate is $f_3 = -(a^2 + b^2) \neq 0$, so the foot is always finite. No slope is used, so vertical and horizontal base lines require no special branch.

A.2 Linear reduction of the three-sphere intersection

For centres c_i and radii r_i , an intersection point x satisfies $\|x - c_i\|^2 = r_i^2$. Subtracting the first equation from the second and third eliminates $\|x\|^2$:

$$2(c_i - c_1)^\top x = r_1^2 - r_i^2 + \|c_i\|^2 - \|c_1\|^2, \quad i \in \{2, 3\}. \quad (41)$$

For non-collinear centres these two planes meet in a line $x = x_0 + t n$, $n \parallel (c_2 - c_1) \times (c_3 - c_1)$. Substituting into the first sphere gives a quadratic in t with zero, one, or two real roots. In the conformal representation, the same reduction is captured by the meet of three spheres, whose result type is already a point pair. The reference implementation uses Equation (41), which provides an independent check of the conformal construction.

A.3 Two-circle intersection formula

Choose coordinates with $O = 0$ and write the target as $T = du$. Decompose an elbow as $E = au + bu_\perp$. The two length equations are

$$a^2 + b^2 = \ell_1^2, \quad (a - d)^2 + b^2 = \ell_2^2. \quad (42)$$

Subtracting them yields $a = (\ell_1^2 - \ell_2^2 + d^2)/(2d)$; substitution gives $b = \pm \sqrt{\ell_1^2 - a^2}$. The two signs are the two point-pair branches. If the radicand is zero, the circles are tangent; if it is negative, the pair is imaginary and the target is unreachable. A short computation shows $\ell_1^2 - a^2 \geq 0$ if and only if Equation (27) holds.

A.4 Proof of Lemma 1

Write $P = \lambda P_0$ with $P_0 = X \wedge Y$ and let $s_0 = -X \cdot Y = \frac{1}{2} \|x - y\|^2 > 0$. For null vectors, $P_0 \cdot P_0 = (X \cdot Y)^2 - X^2 Y^2 = s_0^2$, hence $P \cdot P = \lambda^2 s_0^2$ and $\sqrt{P \cdot P} = |\lambda| s_0$. Moreover $e_\infty \cdot P_0 = (e_\infty \cdot X)Y - (e_\infty \cdot Y)X = X - Y$, and $(X - Y)^2 = -2X \cdot Y = 2s_0$, so $(X - Y)^{-1} = (X - Y)/(2s_0)$. Using $(X \wedge Y)Z = (X \wedge Y) \cdot Z = X(Y \cdot Z) - Y(X \cdot Z)$

for $Z \in \{X, Y\}$ (the outer part vanishes because Z is contained in $X \wedge Y$), we obtain $P_0 X = -s_0 X$ and $P_0 Y = s_0 Y$. Therefore

$$(s_0 + P_0)(X - Y) = s_0 X - s_0 Y - s_0 X - s_0 Y = -2s_0 Y, \quad (-s_0 + P_0)(X - Y) = -2s_0 X, \quad (43)$$

and division by $2s_0$ gives $(\pm s_0 + P_0)(e_\infty \cdot P_0)^{-1} = -Y$ and $-X$, respectively. For $\lambda > 0$ the factors λ cancel and $P_+ = -Y$, $P_- = -X$. For $\lambda < 0$ we have $|\lambda| s_0 + \lambda P_0 = \lambda(-s_0 + P_0)$, so the roles are interchanged: $P_+ = -X$, $P_- = -Y$. In both cases $-P_\pm \cdot e_\infty = -1$, i.e. the weight is -1 . $\square$

A.5 Proof of Proposition 2

Let $O = 0$ and $W = wu$ with $w \neq 0$. Then $\|x - W\|^2 - \|x\|^2 = w^2 - 2wx \cdot u$, so $\text{bis}(0, W) = \{x : x \cdot u = w/2\}$. For $W_+ = (d+1)u$ the bisector is the plane $x \cdot u = (d+1)/2$; its points on $\mathcal{S}(T, 1)$ have axial distance $|(d+1)/2 - d| = |d-1|/2$ from T and hence the transverse radius $h = \sqrt{1 - (d-1)^2/4}$. For $W_- = (d-1)u$, which is non-zero if and only if $d \neq 1$, the bisector is $x \cdot u = (d-1)/2$, and its points on $\mathcal{S}(0, 1)$ have the same transverse radius h . Within the helper plane $\Pi = \text{span}\{u, u_\perp\}$ the upper points are those of Equation (29), so $E_2 - E_1 = u$ and all three links have unit length. The reflection in $\text{bis}(0, T) = \{x : x \cdot u = d/2\}$ maps the axial coordinate $(d+1)/2$ to $d - (d+1)/2 = (d-1)/2$ and preserves the transverse part; hence it maps E_2 to E_1 . The points are real if and only if $(d-1)^2 \leq 4$, i.e. $d \leq 3$. In CGA, the reflection of a conformal point X in the IPNS plane m is $X \mapsto -mXm^{-1}$, and $m = O - T$ represents $\text{bis}(O, T)$ by Equation (17); this gives Equation (30). If the auxiliary points are interchanged, the upper points of $\text{bis}(0, W_+) \cap \mathcal{S}(0, 1)$ and $\text{bis}(0, W_-) \cap \mathcal{S}(T, 1)$ have the transverse radius $\sqrt{1 - (d+1)^2/4}$, which is real only for $d \leq 1$. $\square$

A.6 Error growth near the representation singularity

In floating-point arithmetic the auxiliary point is computed as $\widehat{W}_- = W_- + \eta$ with $\|\eta\| \leq c\varepsilon_{\text{mach}}$ for a modest constant c , because T and u have norms of order one. The bisector of O and $\widehat{W}_-$ has the normal $\widehat{W}_- / \|\widehat{W}_-\|$, whose angle to the exact normal u is of order $\|\eta\| / \|W_-\| = c\varepsilon_{\text{mach}} / |d-1|$, and its offset changes by $O(\varepsilon_{\text{mach}})$. The circle $\text{bis}(O, \widehat{W}_-) \cap \mathcal{S}(O, 1)$ and therefore E_1 move by the same relative order, since E_1 has unit distance from O . By contrast, $\partial E_{1,2} / \partial T$ is bounded near $d = 1$ because $h \approx 1$ there. The forward error $O(\varepsilon_{\text{mach}} / |d-1|)$ is thus caused by the formulation, in agreement with Figure 5c.

B Revised Source Programs

The repository contains the revised CLUCalc programs in `cga_kinematics/` and the revised projective-geometry programs in `projective_geometry/`. For numerical auditing, the behaviour of the original CLUCalc formulations is retained in the functions `tripod_original`, `two_link_original`, and `three_link_original` in `src/cga_constructions.py`. The CLUCalc files below are the revised versions used as the portfolio-facing source programs.

B.1 Hausaufgabe 1: tripod with fourth support

```
1 // Tripod from a three-sphere meet + fourth support
2 // -----
3
4 _BGColor = White;
5 :DRAW_POINT_AS_SPHERE;
6 SetLineWidth(4);
7 DefVarsN3();
8 SetMode(N3_OPNS);
9
10 :U = VecN3(0,0,0) : Black; // origin
11
12 // base points A, B, C of the tripod (ground plane y = 0)
13 :A = VecN3(-1,0,0) : Color(0.35, 0.0, 0.1, 1.0);
14 :B = VecN3( 1,0,0) : Color(0.4, 0.4, 0.2, 1.0);
15 :C = VecN3( 0,0,2) : Color(0.02, 0.02, 0.4, 1.0);
16 :Bodenplatte = 4*(A^B^C^einf) : Color(0.02, 0.02, 0.4, 0.2);
17
18 // one sphere per leg: centre = foot, radius = leg length
19 :Kugel1 = SphereN3(A, 2.69) : Color(0.35, 0.0, 0.1, 0.1);
20 :Kugel2 = SphereN3(B, 1.8) : Color(0.4, 0.4, 0.2, 0.1);
21 :Kugel3 = SphereN3(C, 1.5) : Color(0.02, 0.02, 0.4, 0.1);
22
23 // (1) meet of three spheres = point pair S+ ^ S-
24 S_points = (*Kugel1 ^ *Kugel2 ^ *Kugel3);
25 ?Klass1 = S_points . S_points; // > 0: two real points, 0: tangent, < 0: imaginary
26
27 // (2) factorise the point pair and normalise both points
28 Pp1 = ( sqrt(S_points . S_points) + S_points) / (einf . S_points);
29 Pp2 = (-sqrt(S_points . S_points) + S_points) / (einf . S_points);
30 N_Pp1 = Pp1 / (- (Pp1 . einf));
31 N_Pp2 = Pp2 / (- (Pp2 . einf));
32
33 // (3) policy: the apex is the point with the larger height (y = e2)
34 if (N_Pp1 . e2 > N_Pp2 . e2)
35 {Spitze = N_Pp1}
36 else
37 {Spitze = N_Pp2}
38 :Spitze;
39 ?Spitze;
40
41 // legs
42 DrawLine(Spitze, A);
43 DrawLine(Spitze, B);
44 DrawLine(Spitze, C);
45
46 // (4) fourth leg of length 1.1: its possible feet form the circle
47 // sphere(Spitze, 1.1) ^ ground plane
48 :KugelSpitze = SphereN3(Spitze, 1.1) : Color(1, 1, 0, 0.1);
49 :Schnittkreis = (*Bodenplatte ^ *KugelSpitze) : Black;
50
51 // (5) helper plane tau: vertical plane through A and the apex
52 // (S+ and S- are mirror images, so the plane A^S+^S-^einf is vertical)
53 :Senkrecht_P1 = A ^ N_Pp1 ^ N_Pp2 ^ einf : Color(0.0, 0.3, 0.2, 0.3);
54
55 // (6) circle ^ plane = point pair of candidate feet
56 FussPunkte = (*Senkrecht_P1 ^ *Schnittkreis);
57 ?Klass2 = FussPunkte . FussPunkte;
58 Pp3 = ( sqrt(FussPunkte . FussPunkte) + FussPunkte) / (einf . FussPunkte);
59 Pp4 = (-sqrt(FussPunkte . FussPunkte) + FussPunkte) / (einf . FussPunkte);
60 N_Pp3 = Pp3 / (- (Pp3 . einf));
61 N_Pp4 = Pp4 / (- (Pp4 . einf));
62
63 // (7) policy: the foot farther from A. For this geometry it is the only
64 // candidate that brings the projected apex inside the support polygon
65 // (stability margin +0.304 instead of -0.459, see paper Section 8.2)
66 ?dis3 = sqrt(-2*(A . N_Pp3));
67 ?dis4 = sqrt(-2*(A . N_Pp4));
```

```

68 if (dis3 > dis4)
69 {AussenPunkt = N_Pp3}
70 else
71 {AussenPunkt = N_Pp4}
72 :AussenPunkt;
73 ?AussenPunkt;
74
75 DrawLine(Spitze, AussenPunkt);

```

B.2 Hausaufgabe 2: two-link arm

```

1 // two-link arm, elbow from a circle-plane meet
2 // -----
3
4 _BGColor = White;
5 :DRAW_POINT_AS_SPHERE;
6 SetLineWidth(4);
7 DefVarsN3();
8 SetMode(N3_OPNS);
9
10 // target (the reach of two unit links is 2)
11 ?a = Slider("x", -2, 2, 0.05, 0);
12 ?b = Slider("y", -2, 2, 0.05, 0.5);
13 ?c = Slider("z", -2, 2, 0.05, -1);
14
15 :U = VecN3(0,0,0) : Black; // shoulder
16 :Kugel1 = SphereN3(U,1) : Color(0.05, 0.05, 0.6, 0.3); // upper arm, length 1
17 :A = VecN3(a,b,c) : Black; // target
18 :Kugel2 = SphereN3(A,1) : Color(1, 0, 0, 0.1); // forearm, length 1
19
20 // ground plate (display only)
21 :X = VecN3(2,0,0) : Color(0.05, 0.05, 0.6, 1);
22 :Y = VecN3(0,0,2) : Color(0.05, 0.05, 0.6, 1);
23 :Bodenplatte = 4*(U^X^Y^einf) : Color(0.96, 0.90, 0.70, 0.6);
24
25 // vertical helper plane through U and A
26 if (a*a + c*c > 0.000001)
27 {H = VecN3(0,1,0)}
28 else
29 {H = VecN3(1,0,0)}
30 :E = 8*(U^H^A^einf) : Color(0.0, 0.3, 0.2, 0.3);
31
32 // circle of all possible elbows, then its meet with the helper plane
33 :S_kreis = (*Kugel1 ^ *Kugel2) : Black;
34 S_points = (*S_kreis ^ *E);
35 ?Klass = S_points . S_points; // > 0: reachable, 0: fully stretched, < 0: unreachable
36
37 if (Klass < 0)
38 {
39     ?Unreachable = 1;
40 }
41 else
42 {
43     Pp1 = (sqrt(Klass) + S_points) / (einf . S_points);
44     Pp2 = (-sqrt(Klass) + S_points) / (einf . S_points);
45     N_Pp1 = Pp1 / (-(Pp1 . einf));
46     N_Pp2 = Pp2 / (-(Pp2 . einf));
47
48     // policy: upper elbow (larger y)
49     if (N_Pp1 . e2 > N_Pp2 . e2)
50     {elbow = N_Pp1}
51     else
52     {elbow = N_Pp2}
53     :elbow;
54     ?elbow;
55
56     DrawLine(U, elbow); // upper arm

```

```

57 DrawLine(elbow, A);          // forearm
58 }

```

B.3 Hausaufgabe 3: three-link arm

```

1 // three-link arm U-E1-E2-A with unit links
2 // -----
3
4 _BGColor = White;
5 :DRAW_POINT_AS_SPHERE;
6 SetLineWidth(4);
7 DefVarsN3();
8 SetMode(N3_OPNS);
9
10 // target (the reach of three unit links is 3)
11 ?a = Slider("x", -3, 3, 0.05, 0.5);
12 ?b = Slider("y", -3, 3, 0.05, 0.5);
13 ?c = Slider("z", -3, 3, 0.05, -1);
14
15 :U = VecN3(0,0,0) : Black;          // shoulder
16 :A = VecN3(a,b,c) : Black;          // end point
17 :Kugel1 = SphereN3(U,1) : Color(0.4, 0.4, 0.2, 0.2); // first link (display)
18 :Kugel2 = SphereN3(A,1) : Color(0.02, 0.02, 0.4, 0.2); // last link
19
20 // ground plate (display only)
21 :X = VecN3(2,0,0) : Blue;
22 :Y = VecN3(0,0,2) : Blue;
23 :Bodenplatte = 4*(U^X^Y^einf) : Color(0.35, 0.0, 0.1, 0.3);
24
25 // vertical helper plane through U and A (fallback on the vertical axis)
26 if (a*a + c*c > 0.000001)
27 {H = VecN3(0,1,0)}
28 else
29 {H = VecN3(1,0,0)}
30 :HE = 10*(U^H^A^einf) : Color(0.96, 0.90, 0.70, 0.3);
31
32 // auxiliary points W+- on the line UA at distance 1 from A
33 :Gerade = 4*(U^A^einf);
34 :GKA = (*Gerade ^ *Kugel2);
35 :Pp1 = ( sqrt(GKA . GKA) + GKA) / (einf . GKA);
36 :Pp2 = (-sqrt(GKA . GKA) + GKA) / (einf . GKA);
37 :N_Pp1 = Pp1 / (-Pp1 . einf);
38 :N_Pp2 = Pp2 / (-Pp2 . einf);
39
40 // explicit choice: W+ is the point farther from U (-U.X = |x|^2 / 2)
41 if (-U . N_Pp1) > -(U . N_Pp2)
42 {Wplus = N_Pp1}
43 else
44 {Wplus = N_Pp2}
45 :Wplus;
46
47 // E2: distance 1 from A and equidistant from U and W+
48 :GelenkEbene = *(U - Wplus) : Color(0.5, 0.6, 0.7, 0.3); // bisector plane of U and W+
49 :SKGE = *(GelenkEbene ^ *Kugel2) : Black;
50 :SPP = *(SKGE ^ *HE);
51 ?Klass = SPP . SPP;          // < 0: target out of reach (|A| > 3)
52
53 if (Klass < 0)
54 {
55     ?Unreachable = 1;
56 }
57 else
58 {
59     Pp21 = ( sqrt(Klass) + SPP) / (einf . SPP);
60     Pp22 = (-sqrt(Klass) + SPP) / (einf . SPP);
61     N_Pp21 = Pp21 / (-Pp21 . einf);
62     N_Pp22 = Pp22 / (-Pp22 . einf);

```

```

63
64 // policy: upper configuration
65 if (N_Pp21 . e2 > N_Pp22 . e2)
66 {elbow2 = N_Pp21}
67 else
68 {elbow2 = N_Pp22}
69
70 // E1: reflection of E2 in the bisector plane of U and A
71 M = U - A;
72 elbow1 = -M * elbow2 / M;
73
74 :elbow1;
75 :elbow2;
76 ?elbow1;
77 ?elbow2;
78
79 DrawLine(U, elbow1);
80 DrawLine(elbow1, elbow2);
81 DrawLine(elbow2, A);
82 }

```

B.4 Python programs

The three `libcfcg` programs in `projective_geometry/` implement the revised planar constructions. Their mathematical changes are summarised in Table 7; the core of the revised Aufgabe 4, the locus classification of Proposition 1, reads:

```

1  ux, uy = ux / d, uy / d
2  nx, ny = -uy, ux                                # unit normal of the base
3  h = (p0[0] - p1[0]) * nx + (p0[1] - p1[1]) * ny    # signed height of L//
4  if abs(h) <= REL_TOL * d:
5      return "degenerate: P0 lies on the base line", []
6
7  # parameter t along L//: Q(t) = P1 + t*u + h*n
8  cands = [("bisector", 0.5 * d)]
9  disc = d * d - h * h
10 for label, t0 in (("circle P1", 0.0), ("circle P2", d)):
11     if disc > REL_TOL * d * d:
12         s = math.sqrt(disc)
13         cands += [(label, t0 - s), (label, t0 + s)]
14     elif disc >= -REL_TOL * d * d:
15         cands.append((label + " (tangent)", t0))
16 cands.sort(key=lambda c: c[1])
17 merged = []
18 for label, t in cands:                                # merge coincident apices
19     if merged and abs(t - merged[-1][1]) <= REL_TOL * d:
20         merged[-1] = (merged[-1][0] + " = " + label, merged[-1][1])
21     else:
22         merged.append((label, t))
23 apices = [(lab, (p1[0] + t * ux + h * nx, p1[1] + t * uy + h * ny)) for lab, t in merged]
24 return f"{len(apices)} distinct apices (|h|/d = {abs(h) / d:.4f})", apices
25
26
27 def isosceles_residual(q, p1, p2):
28     a, b, d = math.dist(q, p1), math.dist(q, p2), math.dist(p1, p2)
29     return min(abs(a - b), abs(a - d), abs(b - d)) / d

```

C Reproducibility

The repository contains the manuscript, bibliography, generated figures, numerical results, Python implementation, automated tests, CLUCalc scripts, and the projective-geometry programs. From the repository root,

```
pip install -r requirements-dev.txt
python -m pytest -q
python src/generate_figures.py
cd paper
latexmk -pdf main.tex
```

recreate the automated checks, numerical results, figures, and paper. Setting `FAST=1` before the figure-generation command runs a reduced smoke version of the experiments; only the full run is used for the reported numerical ledger. The file `results/results.json` records every reported value in machine-readable form. The scripts in `projective_geometry/` additionally require the course-specific `libcfcg` teaching library, which is not bundled with the repository. Small last-digit differences may occur when a platform evaluates floating-point operations in a different order.

References

- H. S. M. Coxeter. *Projective Geometry*. Springer, New York, 2 edition, 1987.
- Mark de Berg, Otfried Cheong, Marc van Kreveld, and Mark Overmars. *Computational Geometry: Algorithms and Applications*. Springer, Berlin, 3 edition, 2008. doi: 10.1007/978-3-540-77974-2.
- Leo Dorst, Daniel Fontijne, and Stephen Mann. *Geometric Algebra for Computer Science: An Object-Oriented Approach to Geometry*. Morgan Kaufmann, San Francisco, 2007. ISBN 978-0-12-369465-2.
- Hermann Grassmann. *Die lineale Ausdehnungslehre, ein neuer Zweig der Mathematik*. Otto Wigand, Leipzig, 1844.
- Richard Hartley and Andrew Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, Cambridge, 2 edition, 2004. doi: 10.1017/CBO9780511811685.
- David Hestenes and Garret Sobczyk. *Clifford Algebra to Geometric Calculus: A Unified Language for Mathematics and Physics*. D. Reidel, Dordrecht, 1984. doi: 10.1007/978-94-009-6292-7.
- Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. Society for Industrial and Applied Mathematics, Philadelphia, 2 edition, 2002. doi: 10.1137/1.9780898718027.
- Dietmar Hildenbrand. *Foundations of Geometric Algebra Computing*. Springer, Berlin, 2013. doi: 10.1007/978-3-642-31794-1.
- Dietmar Hildenbrand, Julio Zamora, and Eduardo Bayro-Corrochano. Inverse kinematics computation in computer graphics and robotics using conformal geometric algebra. *Advances in Applied Clifford Algebras*, 18:699–713, 2008. doi: 10.1007/s00006-008-0096-5.
- Adam L. Kleppe and Olav Egeland. Inverse kinematics for industrial robots using conformal geometric algebra. *Modeling, Identification and Control*, 37(1):63–75, 2016. doi: 10.4173/mic.2016.1.6.
- Hongbo Li, David Hestenes, and Alyn Rockwood. Generalized homogeneous coordinates for computational geometry. In Gerald Sommer, editor, *Geometric Computing with Clifford Algebras*. Springer, Berlin, 2001.
- Tobias Löw, Philip Abbet, and Sylvain Calinon. gafro: Geometric algebra for robotics. arXiv preprint arXiv:2310.19090; tutorial version in IEEE Robotics & Automation Magazine, 2023. URL <https://arxiv.org/abs/2310.19090>.
- Richard M. Murray, Zexiang Li, and S. Shankar Sastry. *A Mathematical Introduction to Robotic Manipulation*. CRC Press, Boca Raton, 1994. ISBN 978-0-8493-7981-9.
- Christian Perwass. *Geometric Algebra with Applications in Engineering*. Springer, Berlin, 2009. doi: 10.1007/978-3-540-89068-3.

Jonathan Richard Shewchuk. Adaptive precision floating-point arithmetic and fast robust geometric predicates. *Discrete & Computational Geometry*, 18(3):305–363, 1997. doi: 10.1007/PL00009321.

Isiah Zaplana, Hugo Hadfield, and Joan Lasenby. Closed-form solutions for the inverse kinematics of serial robots using conformal geometric algebra. *Mechanism and Machine Theory*, 173:104835, 2022. doi: 10.1016/j.mechmachtheory.2022.104835.

List of Figures

1	Two uses of homogeneous incidence	6
2	Isosceles apices as line–locus intersections	7
3	Tripod construction and support polygons	10
4	Two-link and three-link constructions	12
5	Geometric and representation singularities	18

List of Tables

1	Source constructions	3
2	Conformal primitives	8
3	Common construction pipeline	13
4	Residual validation	17
5	Circumcentre conditioning	18
6	Execution of the CLUCalc programs in the CGA engine	19
7	Audit of the submitted programs	19